

### Lecture 03:

Recap:

#### 1. Inheritance

class Student:

```
    """Parent class: stores common information for every student."""
```

```
    def __init__(self, name, age, marks):
        self.name = name
        self.age = age
        self.marks = marks
```

```
    def display_basic_info(self):
        print("Name :", self.name)
        print("Age  :", self.age)
        print("Marks:", self.marks)
```

class GraduateStudent(Student):

```
    """Child class: inherits Student and adds specialization."""
```

```
    def __init__(self, name, age, marks, specialization):
        super().__init__(name, age, marks)
        self.specialization = specialization
```

```
    def display_profile(self):
        print("Graduate Student Profile")
        print("-" * 28)
```

```
    # Calling an inherited method from Student
    self.display_basic_info()
```

```
    # Displaying the attribute added in GraduateStudent
    print("Specialization:", self.specialization)
```

# Creating an object of the child class

```
graduate_student = GraduateStudent(
    name="Ali",
    age=20,
    marks=90,
    specialization="Artificial Intelligence"
)
```

# Calling the child-class method

```
graduate_student.display_profile()
```

2. Numpy: declare rank 2 matrix instead rank1, Vectorization: SIMD, and broadcasting

## Today Lecture:

### Numpy

#### 1. Accessing of numpy arrays:

```
import numpy as np
b = np.array([[1,2,3],[4,5,6]])    # Create a rank 2 array
print(b.shape)
print(b[0, 0], b[0, 1], b[1, 0])  # Prints "1 2 4"
#OR like C/C++
print(b[0][0], b[0][1], b[1][0])  # Prints "1 2 4"
```

#### 2. Slicing: View vs copy in slicing

If you want b to be independent of a, explicitly make a copy:

```
import numpy as np
a = np.array([[1,2,3,4], [5,6,7,8], [9,10,11,12]])
#b = a # This is a *view* is a shallow copy
b = a[:2, 1:3]    # This is a *view*
b[0,0] = 77      # Modifies `a` as well
print(a)
```

```
import numpy as np
```

```
a = np.array([[1,2,3,4], [5,6,7,8], [9,10,11,12]])
```

```
b = a[:2, 1:3].copy()    # Make a deep copy
b[0,0] = 77              # Modify b only
```

```
print("a:\n", a)    # Unchanged
print("b:\n", b)    # Changed
```

Note: When you index into numpy arrays using slicing, the resulting array view will always be a subarray of the original array.

## Integer vs Boolean indexing

1. Integer array indexing: When you index into numpy arrays using slicing, the resulting array view will always be a subarray of the original array.
2. In contrast, integer array indexing allows you to construct arbitrary arrays using the data from another array. Here is an example:

```
import numpy as np
# rank 2
a = np.array([[1,2], [3, 4], [5, 6]])
print (a.shape)

#created by interger index of rank 1 array
print("-" * 28)
print ("Rank 1 arrays")
b = np.array(a[[0, 1, 2],[0, 1, 0]] )
print (b.shape)
# OR, both syntax are equivalent
b = np.array([a[0][0], a[1][1], a[2][0]] )
print (b.shape)
b= np.array([a[0, 0], a[1, 1], a[2, 0]]) # Prints "[1 4 5]"
print (b.shape)

#[] outside []make the return number as a list that is given
# as input to np.array()
print("-" * 28)
print ("Rank 2 arrays")
# add []outside, created rank 2 array
b = np.array([a[[0, 1, 2],[0, 1, 0]]] )
print (b.shape)
# OR, both syntax are equivalent
b = np.array([[a[0][0], a[1][1], a[2][0]]] )
print (b.shape)
b= np.array([[a[0, 0], a[1, 1], a[2, 0]]]) # Prints "[1 4 5]"
print (b.shape)
```

One useful trick with integer array indexing is selecting or mutating one element from each row of a matrix:

```
import numpy as np

# Create a new array from which we will select elements
a = np.array([[1,2,3], [4,5,6], [7,8,9], [10, 11, 12]])
print (a)
#print (a.shape)

#print the specific elements
print (a[np.arange(4),b]) # select specific element
#mutate the specific elements
a[np.arange(4),b] = a[np.arange(4),b] + 10
#OR a[np.arange(4), b] += 10
print ("-" * 25)
print ("mutate array:")
print (a)
```

Boolean array indexing: Boolean array indexing lets you pick out arbitrary elements of an array. Frequently this type of indexing is used to select the elements of an array that satisfy some condition. Here is an example:

```
import numpy as np

# Create a new array from which we will select elements
a = np.array([[1,2,3], [4,5,6], [7,8,9], [10, 11, 12]])
print (a)
#print (a.shape)
boolArray = a > 2

print (a[boolArray])
# [[False False  True]
# [ True  True  True]
# [ True  True  True]
# [ True  True  True]]

#OR
print ("-" * 28)
print (a[a > 2]) # [ 3  4  5  6  7  8  9 10 11 12]
```

;;;;;

## i. Datatypes

Every numpy array is a grid of elements of the same type. Numpy provides a large set of numeric datatypes that you can use to construct arrays. Numpy tries to guess a datatype when you create an array, but functions that construct arrays usually also include an optional argument to explicitly specify the datatype. Here is an example:

```
import numpy as np
x = np.array([1, 2])    # Let numpy choose the datatype
print(x.dtype)         # Prints "int64"

x = np.array([1.0, 2.0]) # Let numpy choose the datatype
print(x.dtype)          # Prints "float64"

x = np.array([1, 2], dtype=np.int32) # Force a particular datatype
print(x.dtype)
```

## ii. Array math

Basic mathematical functions operate elementwise on arrays, and are available both as operator overloads and as functions in the numpy module:

```
import numpy as np

x = np.array([[1,2],[3,4]], dtype=np.float64)
y = np.array([[5,6],[7,8]], dtype=np.float64)

# Elementwise sum; both produce the array
# [[ 6.0  8.0]
#  [10.0 12.0]]
print(x + y)
print(np.add(x, y))

# Elementwise difference; both produce the array
# [[-4.0 -4.0]
#  [-4.0 -4.0]]
```

```

print(x - y)
print(np.subtract(x, y))

# Elementwise product; both produce the array
# [[ 5.0 12.0]
#  [21.0 32.0]]
print(x * y)
print(np.multiply(x, y))

# Elementwise division; both produce the array
# [[ 0.2          0.33333333]
#  [ 0.42857143  0.5         ]]
print(x / y)
print(np.divide(x, y))

# Elementwise square root; produces the array
# [[ 1.          1.41421356]
#  [ 1.73205081  2.         ]]
print(np.sqrt(x))

```

Note that unlike MATLAB, `*` is elementwise multiplication, not matrix multiplication. We instead use the `dot` function to compute inner products of vectors, to multiply a vector by a matrix, and to multiply matrices. `dot` is available both as a function in the `numpy` module and as an instance method of array objects:

```

import numpy as np
v = np.array([9,10])
w = np.array([11, 12])
print (v.shape) # (2,)
print(v.shape) # (2,)

# Inner product of vectors; both produce 219
print( v.dot(w))
print(np.dot(v, w))

x = np.array([[1,2],[3,4]])
print (x.shape)
print(np.dot(x, v)) # here v is act as column vector.
# inorder to remove ambiguity specify v as rank 2 matrix

```

Numpy provides many useful functions for performing computations on arrays; one of the most useful is sum:

```
import numpy as np
x = np.array([[1,2],[3,4]])

print(np.sum(x)) # Compute sum of all elements; prints "10"
print((np.sum(x, axis=0)).shape) # Compute sum of each column; prints
"[4 6]"
print(np.sum(x, axis=1)) # Compute sum of each row; prints "[3 7]"
```

## OpenCV and matplotlib:

```
import matplotlib.pyplot as plt
import cv2 as cv
import numpy as np
image = cv.imread('cat.png')
image = cv.cvtColor(image, cv.COLOR_BGR2RGB)
print(image.dtype, image.shape, type(image)) # Prints "uint8 (400, 248, 3)"

print (image.min())
print (image.max())
img_tinted = image * [1, 0.95, 0.9] # Broadcasting makes 3 2D arrays of size
                                     # 400x248 having values 1, 0.95, and 0.9
                                     # in each 2D array and then perform
                                     # elementwise multiplication operation

img_tinted = cv.resize(img_tinted, (200, 100)) # (width, height)

print (img_tinted.min())
print (img_tinted.max())

# Show the original image
plt.subplot(1, 2, 1)
plt.imshow(image)

# Show the tinted image
plt.subplot(1, 2, 2)

# A slight gotcha with imshow is that it might give strange results
# if presented with data that is not uint8. To work around this, we
# explicitly cast the image to uint8 before displaying it.
plt.imshow(np.uint8(img_tinted))
plt.show()
```

